

Laboratorium 9.1 — bagging i lasy losowe

Statystyka w SI i analizie danych — ćwiczenia

Wojciech Maćkowiak, 412430

2026-05-05

Contents

Regresja — Boston 1

Klasyfikacja — CarseatsH 9

Podsumowanie 15

Pakiety: randomForest, MASS, ISLR, plotmo. **Instalacja:** `install.packages(c("randomForest", "MASS", "ISLR", "plotmo"))`.

```
library(randomForest)
library(MASS)
library(ISLR)
library(plotmo)
```

Regresja — Boston

Otwarta implementacja lasów losowych — i baggingu jako przypadku szczególnego (`mtry` równe liczbie predyktorów) — jest w pakiecie `randomForest`.

```
head(Boston)
```

```
##      crim zn indus chas   nox    rm  age    dis rad tax ptratio  black lstat
## 1 0.00632 18  2.31    0 0.538 6.575 65.2 4.0900   1 296    15.3 396.90  4.98
## 2 0.02731  0  7.07    0 0.469 6.421 78.9 4.9671   2 242    17.8 396.90  9.14
## 3 0.02729  0  7.07    0 0.469 7.185 61.1 4.9671   2 242    17.8 392.83  4.03
## 4 0.03237  0  2.18    0 0.458 6.998 45.8 6.0622   3 222    18.7 394.63  2.94
## 5 0.06905  0  2.18    0 0.458 7.147 54.2 6.0622   3 222    18.7 396.90  5.33
## 6 0.02985  0  2.18    0 0.458 6.430 58.7 6.0622   3 222    18.7 394.12  5.21
##   medv
## 1 24.0
## 2 21.6
## 3 34.7
## 4 33.4
## 5 36.2
## 6 28.7
```

Las losowy z domyślną liczbą podziałów (dla regresji $mtry \approx \sqrt{p}$). Parametr `importance = TRUE` włącza liczenie ważności predyktorów.

```

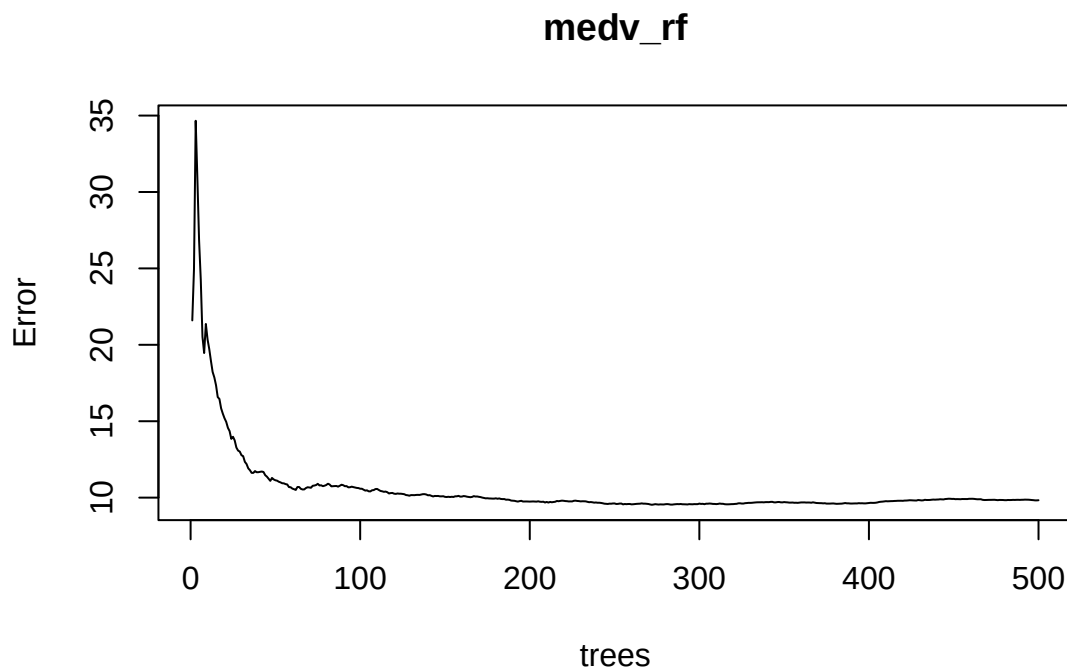
medv_rf <- randomForest(medv ~ ., data = Boston, importance = TRUE)
medv_rf

##
## Call:
## randomForest(formula = medv ~ ., data = Boston, importance = TRUE)
##           Type of random forest: regression
##           Number of trees: 500
## No. of variables tried at each split: 4
##
##           Mean of squared residuals: 9.828467
##           % Var explained: 88.36

```

Wykres błędu OOB względem liczby drzew.

```
plot(medv_rf, type = "l")
```



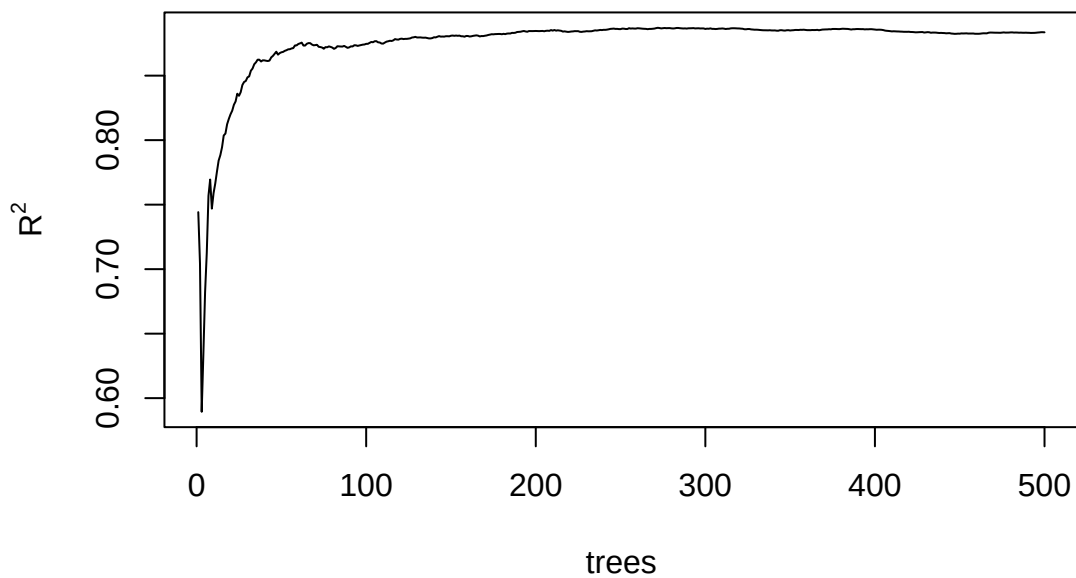
Dla regresji składowa mse zawiera błąd MSE OOB; ostatnie wartości:

```
tail(medv_rf$mse)
```

```
## [1] 9.854631 9.843417 9.833823 9.820568 9.820282 9.828467
```

Składowa rsq to pseudo- R^2 liczone z MSE OOB i wariancji odpowiedzi.

```
plot(medv_rf$rsq, type = "l", xlab = "trees", ylab = expression(R^2))
```



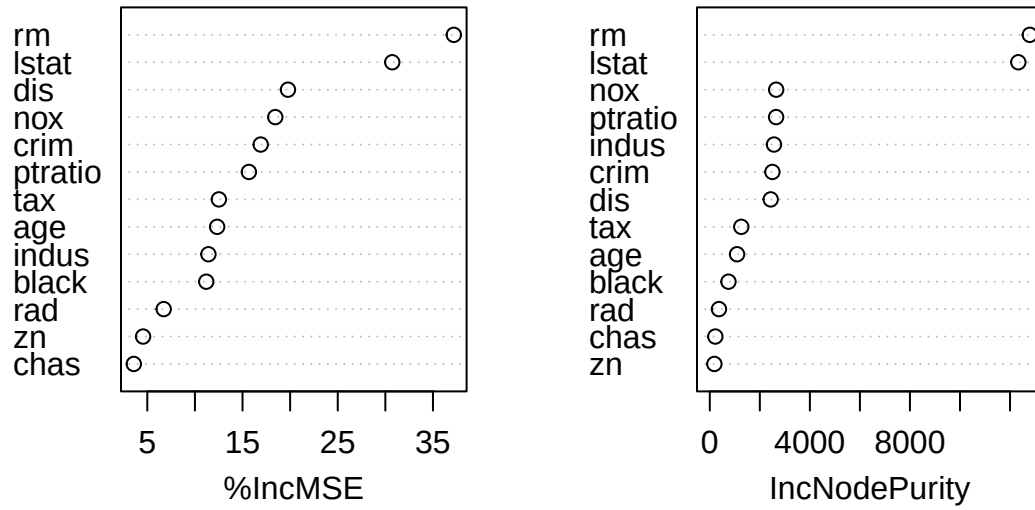
Ważność predyktorów (%IncMSE, IncNodePurity):

```
importance(medv_rf)
```

##		%IncMSE	IncNodePurity
##	crim	16.908986	2502.2114
##	zn	4.554837	182.0693
##	indus	11.417858	2565.7741
##	chas	3.581630	221.5839
##	nox	18.440161	2652.5874
##	rm	37.188782	12790.8893
##	age	12.335582	1088.8578
##	dis	19.771186	2433.5093
##	rad	6.721276	365.8886
##	tax	12.516781	1258.4335
##	ptratio	15.668117	2650.6696
##	black	11.193363	746.3955
##	lstat	30.725051	12332.4710

```
varImpPlot(medv_rf)
```

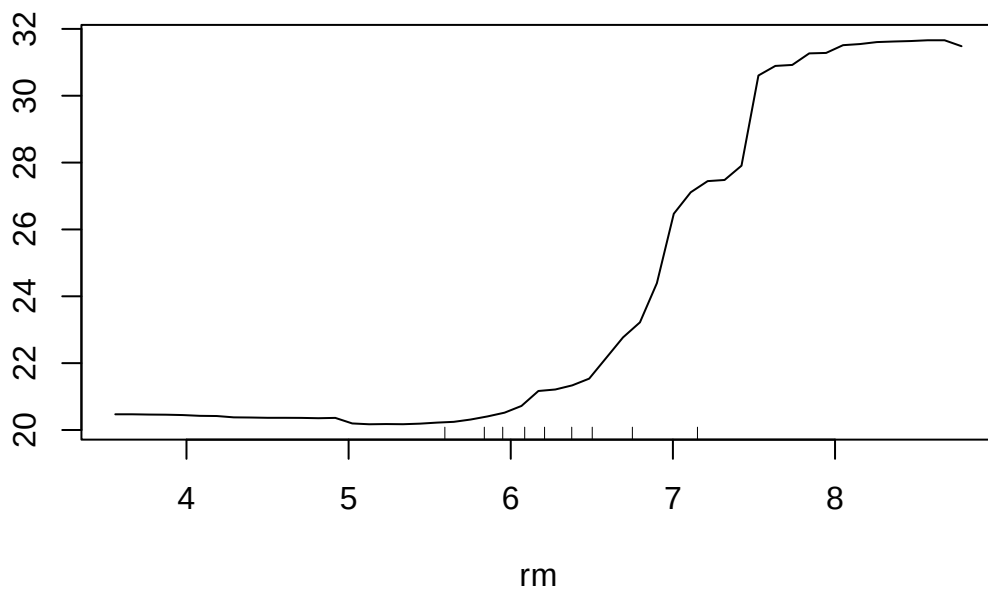
medv_rf



Wykresy częściowej zależności dla wybranych zmiennych:

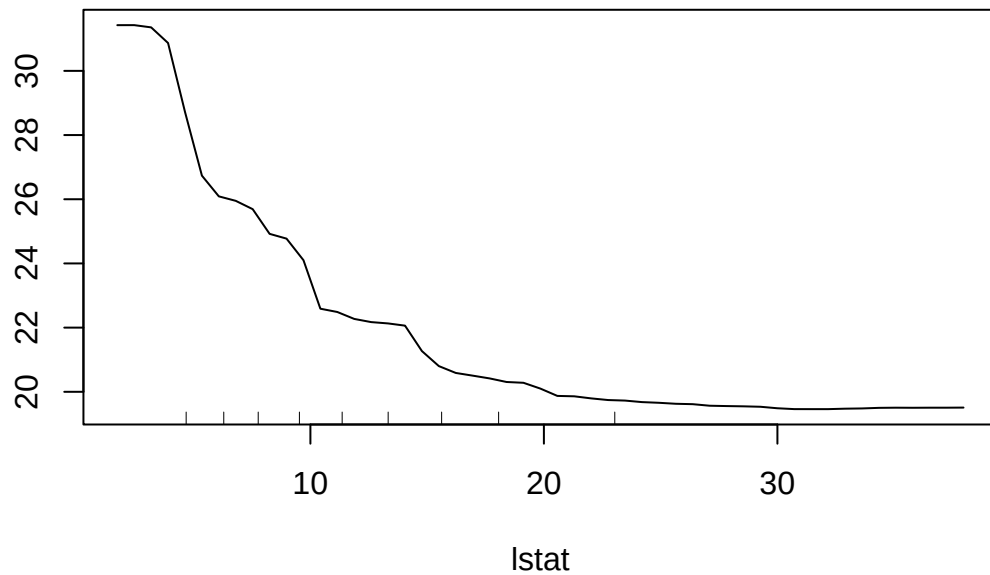
```
partialPlot(medv_rf, Boston, x.var = rm)
```

Partial Dependence on rm



```
partialPlot(medv_rf, Boston, x.var = lstat)
```

Partial Dependence on lstat



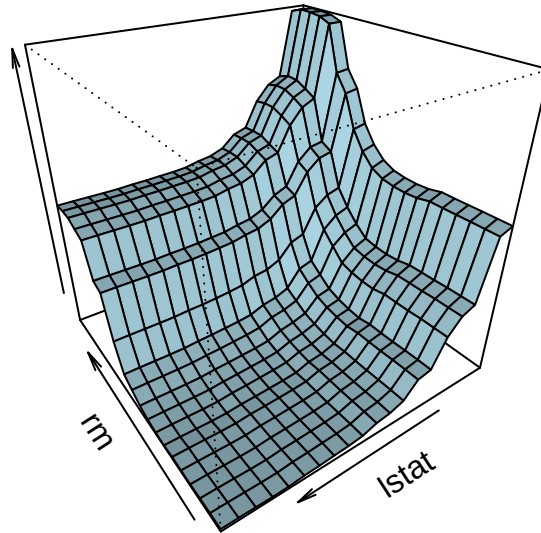
Funkcja `partialPlot()` z `randomForest` to uproszczone PDP; `plotmo` pozwala m.in. na interakcje.

```
plotmo(medv_rf, pmethod = "partdep", degree1 = FALSE, degree2 = c("rm", "lstat"))
```

```
## calculating partdep for rm:lstat 01234567890
```

```
medv ~ randomForest(medv~., data=Boston, import...
```

rm: lstat



Model baggingowy (mtry = 13 — wszystkie predyktory przy każdym podziale):

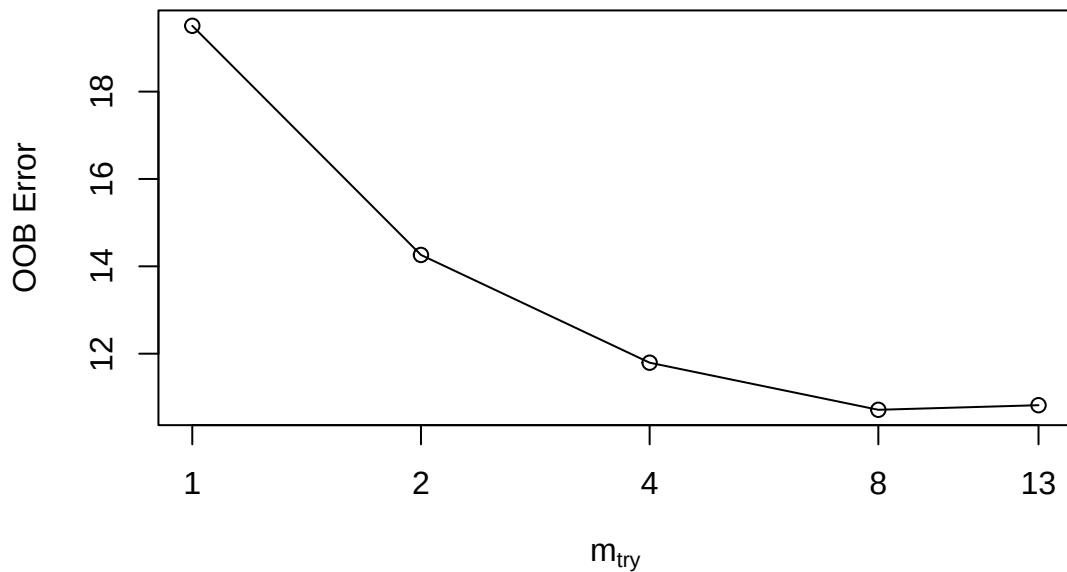
```
medv_bag <- randomForest(medv ~ ., data = Boston, mtry = 13, importance = TRUE)
medv_bag
```

```
##
## Call:
## randomForest(formula = medv ~ ., data = Boston, mtry = 13, importance = TRUE)
##           Type of random forest: regression
##           Number of trees: 500
## No. of variables tried at each split: 13
##
##           Mean of squared residuals: 10.4399
##           % Var explained: 87.63
```

Dobór mtry funkcją tuneRF (start od 2, krok $\times 2$):

```
tune_res <- tuneRF(Boston[, -14], Boston[, 14], mtryStart = 2, stepFactor = 2, trace = FALSE)

## -0.367843 0.05
## 0.173007 0.05
## 0.09141697 0.05
## -0.009790668 0.05
```



```
tune_res
```

```
##      mtry OOBError
## 1         1 19.50699
## 2         2 14.26113
## 4         4 11.79386
## 8         8 10.71570
## 13        13 10.82061
```

```
medv_rf_tuned <- NULL
row_min <- which.min(tune_res[, "OOBError"])
mtry_min <- tune_res[row_min, "mtry"]

if (!(mtry_min %in% c(medv_rf$mtry, 13))) {
  medv_rf_tuned <- randomForest(medv ~ ., data = Boston, mtry = mtry_min)
  medv_rf_tuned
} else if (mtry_min == medv_rf$mtry) {
  "Równa domyślnej"
} else {
  "Równa maksymalnej"
}
```

```
##
## Call:
## randomForest(formula = medv ~ ., data = Boston, mtry = mtry_min)
##              Type of random forest: regression
##              Number of trees: 500
## No. of variables tried at each split: 8
##
##              Mean of squared residuals: 9.934938
```

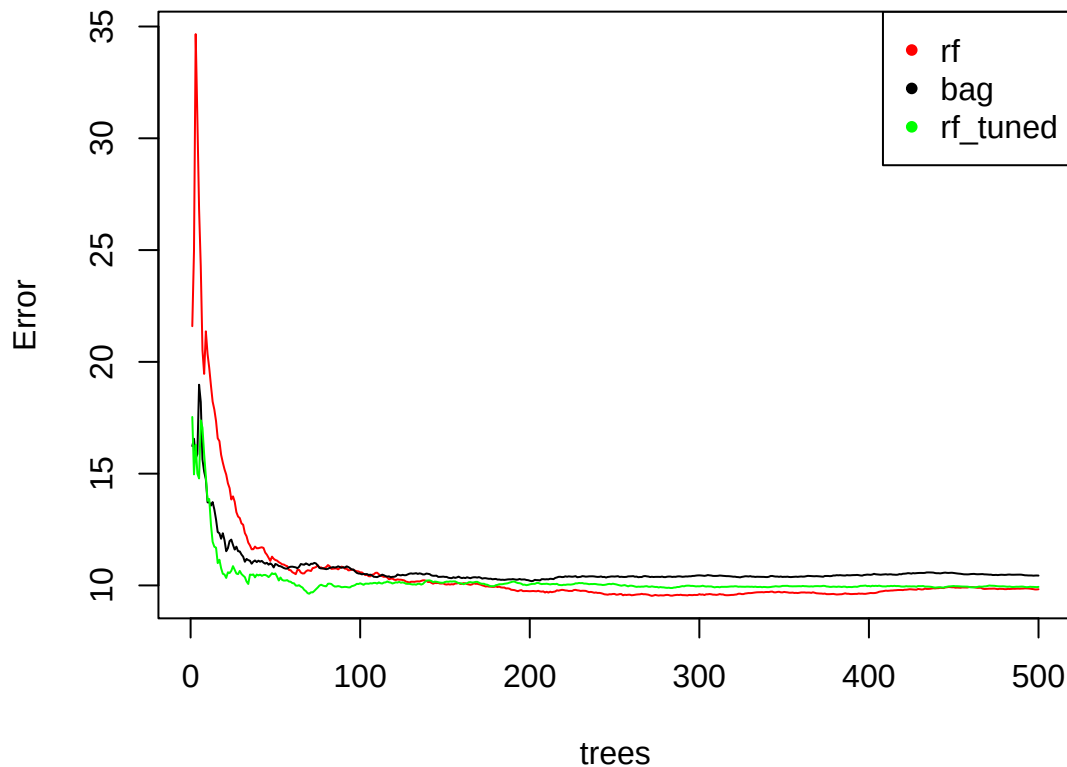
```
## % Var explained: 88.23
```

Porównanie krzywych MSE OOB: las domyślny, bagging, ewentualnie las strojony.

```
cols <- c("red", "black", "green")

plot(medv_rf, type = "l", col = cols[1], main = NULL)
lines(medv_bag$mse, col = cols[2])
lgd <- c("rf", "bag")

if (!is.null(medv_rf_tuned)) {
  lines(medv_rf_tuned$mse, col = cols[3])
  lgd <- c(lgd, "rf_tuned")
}
legend("topright", legend = lgd, col = head(cols, length(lgd)), pch = 20)
```



Oszacowanie błędu testowego — ten sam podział co w laboratorium 8 (`set.seed(2025)`, indeksy `train_bo` / `test_bo`).

```
set.seed(2025)
n_bo <- nrow(Boston)
train_bo <- sample(n_bo, 0.8 * n_bo)
test_bo <- setdiff(seq_len(n_bo), train_bo)

medv_rf_subset <- randomForest(medv ~ ., data = Boston, subset = train_bo)
```



```

medv_rf_subset_pred <- predict(medv_rf_subset, newdata = Boston[test_bo, ])
mean((medv_rf_subset_pred - Boston$medv[test_bo])^2)

## [1] 11.16354

medv_bag_subset <- randomForest(medv ~ ., data = Boston, subset = train_bo, mtry = 13)
medv_bag_subset_pred <- predict(medv_bag_subset, newdata = Boston[test_bo, ])
mean((medv_bag_subset_pred - Boston$medv[test_bo])^2)

## [1] 11.44659

if (!is.null(medv_rf_tuned)) {
  medv_tuned_subset <- randomForest(medv ~ ., data = Boston, subset = train_bo, mtry = mtry_min)
  medv_tuned_subset_pred <- predict(medv_tuned_subset, newdata = Boston[test_bo, ])
  mean((medv_tuned_subset_pred - Boston$medv[test_bo])^2)
}

## [1] 10.36373

```

Klasyfikacja — CarseatsH

```

CarseatsH <-
  Carseats |>
  transform(High = factor(ifelse(Sales <= 8, "No", "Yes")))
head(CarseatsH)

```

```

##   Sales CompPrice Income Advertising Population Price ShelfLoc Age Education
## 1  9.50      138     73          11         276   120      Bad  42         17
## 2 11.22      111     48          16         260    83     Good  65         10
## 3 10.06      113     35          10         269    80   Medium  59         12
## 4  7.40      117    100           4         466    97   Medium  55         14
## 5  4.15      141     64           3         340   128     Bad  38         13
## 6 10.81      124    113          13         501    72     Bad  78         16
##   Urban  US High
## 1   Yes Yes  Yes
## 2   Yes Yes  Yes
## 3   Yes Yes  Yes
## 4   Yes Yes   No
## 5   Yes No   No
## 6    No Yes  Yes

```

Las losowy z domyślnym mtry (dla klasyfikacji $p/3$).

```

csh_rf <- randomForest(High ~ . - Sales, data = CarseatsH, importance = TRUE)
csh_rf

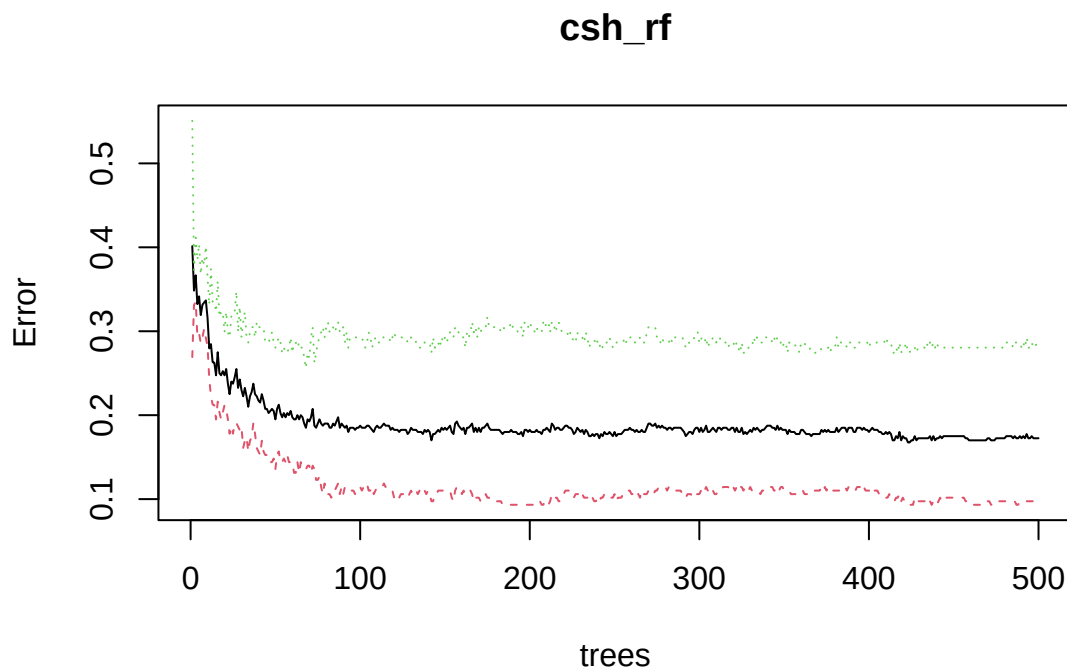
##
## Call:
## randomForest(formula = High ~ . - Sales, data = CarseatsH, importance = TRUE)
##               Type of random forest: classification
##               Number of trees: 500
## No. of variables tried at each split: 3
##
##               OOB estimate of  error rate: 17.25%

```

```
## Confusion matrix:
##      No Yes class.error
## No  214  22  0.09322034
## Yes  47 117  0.28658537
```

Wykres błędu OOB — czarna: ogólny OOB, zielona: klasa Yes, czerwona: No.

```
plot(csh_rf, type = "l")
```



```
tail(csh_rf$err.rate)
```

```
##      OOB      No      Yes
## [495,] 0.1725 0.09745763 0.2804878
## [496,] 0.1750 0.09745763 0.2865854
## [497,] 0.1725 0.09745763 0.2804878
## [498,] 0.1725 0.09745763 0.2804878
## [499,] 0.1725 0.09322034 0.2865854
## [500,] 0.1725 0.09322034 0.2865854
```

```
csh_rf$confusion
```

```
##      No Yes class.error
## No  214  22  0.09322034
## Yes  47 117  0.28658537
```

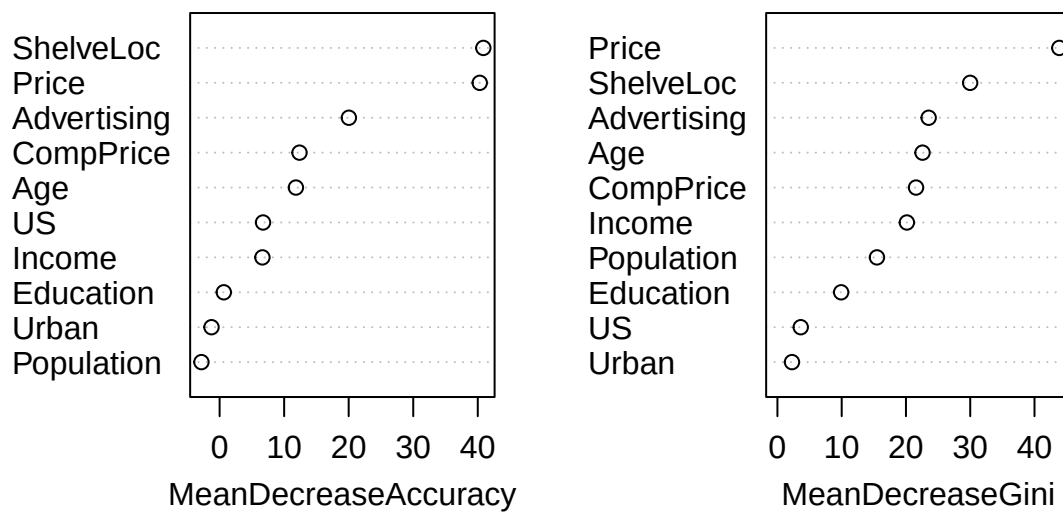
```
importance(csh_rf)
```

```
##      No      Yes MeanDecreaseAccuracy MeanDecreaseGini
## CompPrice 10.396342 7.860517      12.3807141      21.567201
## Income     4.399176 5.067917      6.6310765      20.141235
## Advertising 11.346704 17.013897     20.0282505     23.529995
## Population -1.899196 -2.061748     -2.8164619     15.483887
```

```
## Price      31.229719 30.197803      40.3048736      43.861617
## ShelfLoc   30.305845 35.244333      40.8601430      29.984668
## Age        8.042250 9.635719      11.8213483      22.575685
## Education  1.877105 -1.342599      0.6484435       9.914560
## Urban      0.279699 -2.231064     -1.2556213       2.263519
## US         1.846683 6.076241      6.7276509       3.628068
```

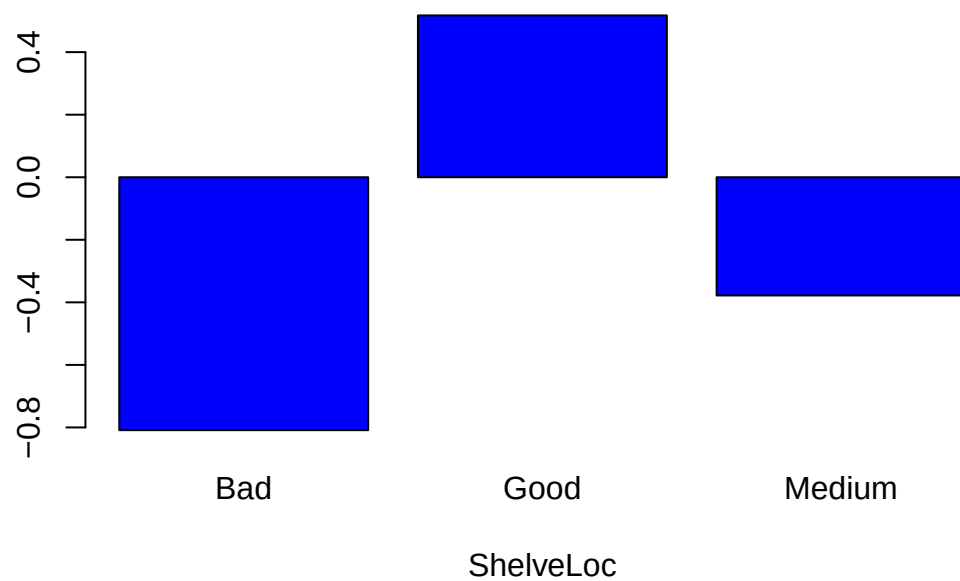
```
varImpPlot(csh_rf)
```

csh_rf



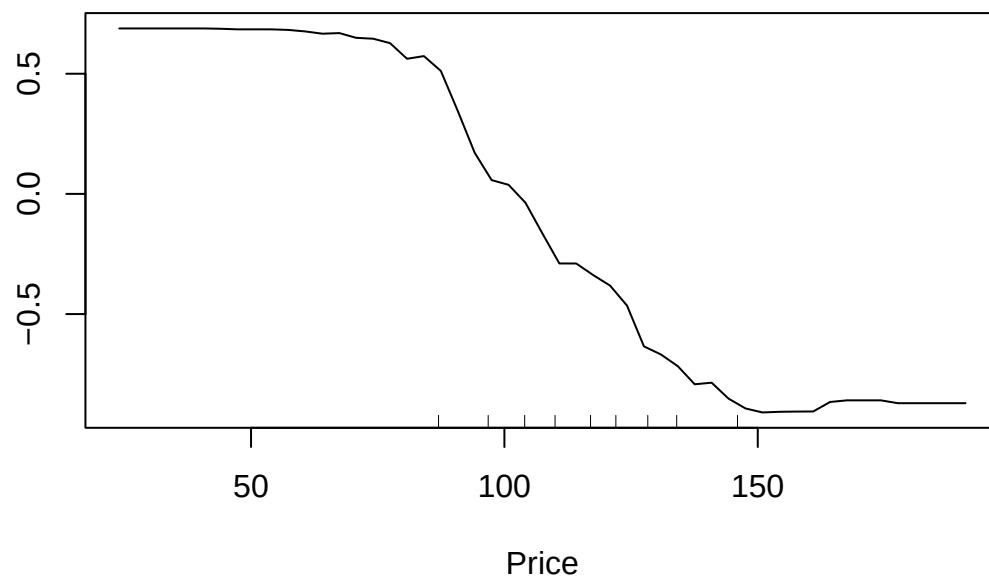
```
partialPlot(csh_rf, CarseatsH, x.var = ShelveLoc, which.class = "Yes")
```

Partial Dependence on ShelfLoc



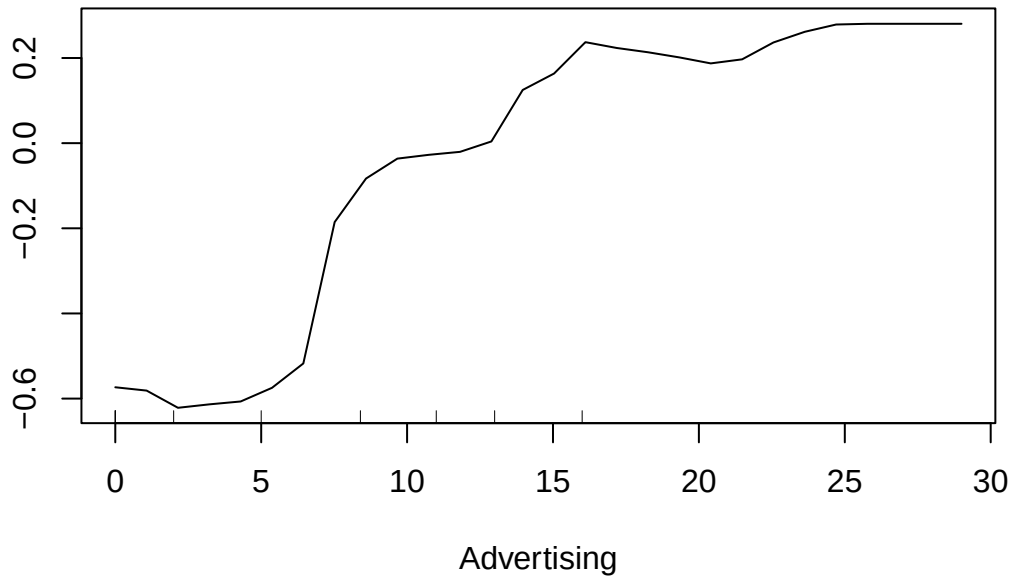
```
partialPlot(csh_rf, CarseatsH, x.var = Price, which.class = "Yes")
```

Partial Dependence on Price



```
partialPlot(csh_rf, CarseatsH, x.var = Advertising, which.class = "Yes")
```

Partial Dependence on Advertising

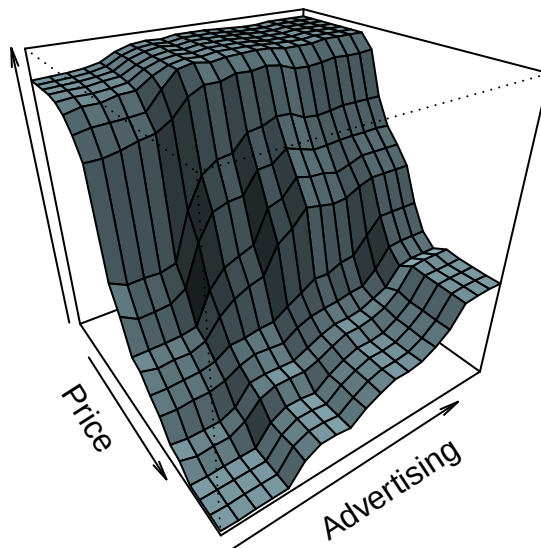


```
plotmo(csh_rf, pmethod = "partdep", degree2 = c("^Price", "Advertising"))
```

```
## calculating partdep for Price:Advertising 01234567890
```

```
randomForest(High~.-Sales, data=CarseatsH, impor...
```

Price: Advertising



Bagging (mtry = 10 — liczba predyktorów w modelu):

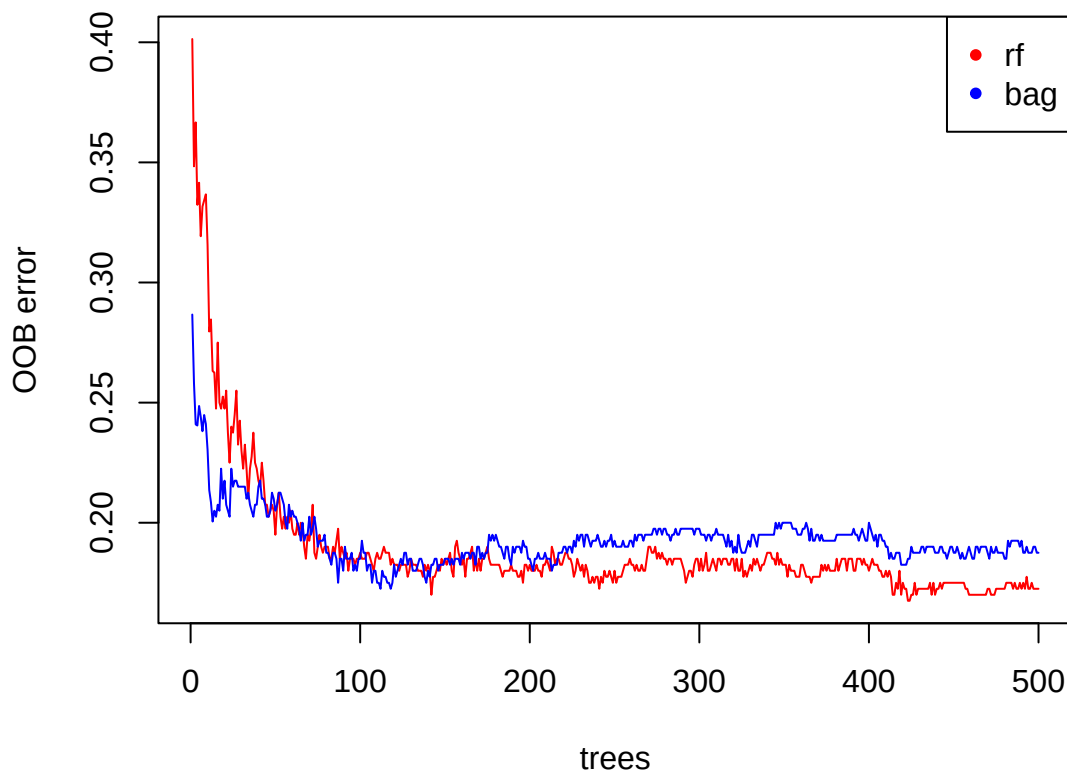
```

csh_bag <- randomForest(High ~ . - Sales, data = CarseatsH, mtry = 10, importance = TRUE)
csh_bag

##
## Call:
## randomForest(formula = High ~ . - Sales, data = CarseatsH, mtry = 10,      importance = TRUE)
##           Type of random forest: classification
##           Number of trees: 500
## No. of variables tried at each split: 10
##
##           OOB estimate of  error rate: 18.75%
## Confusion matrix:
##           No Yes class.error
## No  206  30   0.1271186
## Yes  45 119   0.2743902

cols_oob <- c("red", "blue")
plot(csh_rf$err.rate[, "OOB"], type = "l", col = cols_oob[1], xlab = "trees", ylab = "OOB error")
lines(csh_bag$err.rate[, "OOB"], type = "l", col = cols_oob[2])
legend("topright", legend = c("rf", "bag"), col = cols_oob, pch = 20)

```



Błąd klasyfikacji na zbiorze testowym (set.seed(1), indeksy train_cs / test_cs).

```

set.seed(1)
n_cs <- nrow(CarseatsH)
train_cs <- sample(n_cs, 0.8 * n_cs)
test_cs <- setdiff(seq_len(n_cs), train_cs)

csh_rf_subset <- randomForest(High ~ . - Sales, data = CarseatsH, subset = train_cs)
csh_rf_subset_pred <- predict(csh_rf_subset, newdata = CarseatsH[test_cs, ])
mean(csh_rf_subset_pred != CarseatsH$High[test_cs])

## [1] 0.1875

csh_bag_subset <- randomForest(High ~ . - Sales, data = CarseatsH, subset = train_cs, mtry = 10)
csh_bag_subset_pred <- predict(csh_bag_subset, newdata = CarseatsH[test_cs, ])
mean(csh_bag_subset_pred != CarseatsH$High[test_cs])

## [1] 0.15

```

Podsumowanie

Zastosowano `randomForest` do regresji (`Boston`) i klasyfikacji (`CarseatsH`): las domyślny, bagging, opcjonalny dobór `mtry`, ważność zmiennych, PDP oraz błąd na holdoutcie przy ujednoliconym podziale train/test jak w laboratorium 8.